```
#Libraries
!pip install matplotlib rouge_score transformers sentencepiece
```

```
Requirement already satisfied: matplotlib in
/usr/local/lib/python3.12/dist-packages (3.10.0)
Collecting rouge_score
  Downloading rouge_score-0.1.2.tar.gz (17 kB)
  Preparing metadata (setup.py) ... ent already satisfied:
transformers in /usr/local/lib/python3.12/dist-packages (4.57.3)
Requirement already satisfied: sentencepiece in
/usr/local/lib/python3.12/dist-packages (0.2.1)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: numpy>=1.23 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (2.0.2)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.12/dist-packages (from matplotlib)
(2.9.0.post0)
Requirement already satisfied: absl-py in
/usr/local/lib/python3.12/dist-packages (from rouge_score) (1.4.0)
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-
packages (from rouge_score) (3.9.1)
Requirement already satisfied: six>=1.14.0 in
/usr/local/lib/python3.12/dist-packages (from rouge_score) (1.17.0)
Requirement already satisfied: filelock in
/usr/local/lib/python3.12/dist-packages (from transformers) (3.20.0)
Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in
/usr/local/lib/python3.12/dist-packages (from transformers) (0.36.0)
Requirement already satisfied: pyyaml>=5.1 in
/usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in
/usr/local/lib/python3.12/dist-packages (from transformers)
(2025.11.3)
Requirement already satisfied: requests in
/usr/local/lib/python3.12/dist-packages (from transformers) (2.32.4)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in
/usr/local/lib/python3.12/dist-packages (from transformers) (0.22.1)
Requirement already satisfied: safetensors>=0.4.3 in
```

```
/usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in
/usr/local/lib/python3.12/dist-packages (from transformers) (4.67.1)
Requirement already satisfied: fsspec>=2023.5.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<1.0,>=0.34.0->transformers) (2025.3.0)
Requirement already satisfied: typing-extensions>=3.7.4.3 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<1.0,>=0.34.0->transformers) (4.15.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<1.0,>=0.34.0->transformers) (1.2.0)
Requirement already satisfied: click in
/usr/local/lib/python3.12/dist-packages (from nltk->rouge_score)
(8.3.1)
Requirement already satisfied: joblib in
/usr/local/lib/python3.12/dist-packages (from nltk->rouge_score)
(1.5.3)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests->transformers)
(3.4.4)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.12/dist-packages (from requests->transformers)
(3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests->transformers)
(2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests->transformers)
(2025.11.12)
Building wheels for collected packages: rouge_score
  Building wheel for rouge_score (setup.py) ... e=rouge_score-0.1.2-
py3-none-any.whl size=24934
sha256=44455efcfce5fc0afcb53e037bbdd55ebf0f5b4893bcd705af8b18bcc9af011
5
  Stored in directory:
/root/.cache/pip/wheels/85/9d/af/01feefbe7d55ef5468796f0c68225b6788e85
d9d0a281e7a70
Successfully built rouge_score
Installing collected packages: rouge_score
Successfully installed rouge_score-0.1.2
```

```python
#Imports
import re
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.models import Model
from tensorflow.keras.callbacks import EarlyStopping
```

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.layers import Input, Embedding, LSTM, Dense
```

50000: Limits the number of unique words to reduce model size and memory usage.

400: Fixes the maximum length of input articles for uniform processing.

50: Sets the maximum length of generated summaries.

128: Controls the size of word embeddings to capture semantic meaning efficiently.

128: Defines the number of hidden units in the LSTM for sequence learning.

20: Specifies how many times the model is trained on the dataset.

64: Determines how many samples are processed together during training.

```
#Config
max_vocab = 50000
max_article_len = 400
max_summary_len = 50
embedding_dim = 128
latent_dim = 128
epochs = 20
batch_size = 64

#Load dataset
train_df = pd.read_csv('train.csv')
val_df   = pd.read_csv('validation.csv')
test_df  = pd.read_csv('test.csv')
```

This code performs text preprocessing on the dataset. All text is converted to lowercase, special characters are removed, and extra spaces are eliminated to ensure clean and consistent input for the model. Additionally, special tokens startseq and endseq are added to summaries to mark the beginning and end of the target sequence, which helps the decoder learn when to start and stop generating summaries. Lowercasing and removing special characters helps reduce vocabulary size, remove noise, and make learning more efficient for the model.

```
#Cleaning text
def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^a-zA-Z]', ' ', text)
    text = re.sub(r'\s+', ' ', text)
    return text

train_articles = train_df['article'].astype(str).apply(clean_text)
train_summaries = train_df['highlights'].astype(str).apply(clean_text)

val_articles = val_df['article'].astype(str).apply(clean_text)
```

```
val_summaries = val_df['highlights'].astype(str).apply(clean_text)

test_articles = test_df['article'].astype(str).apply(clean_text)
test_summaries = test_df['highlights'].astype(str).apply(clean_text)

train_summaries = 'startseq ' + train_summaries + ' endseq'
val_summaries   = 'startseq ' + val_summaries + ' endseq'
test_summaries  = 'startseq ' + test_summaries + ' endseq'
```

The dataset size is limited to balance training time, GPU memory usage, and model performance. A 30k–3k–3k split balances learning quality, evaluation reliability, and computational efficiency.

```
#Limiting dataset size
N_TRAIN = 30000
N_VAL   = 3000
N_TEST  = 3000

train_articles = train_articles[:N_TRAIN]
train_summaries = train_summaries[:N_TRAIN]

val_articles = val_articles[:N_VAL]
val_summaries = val_summaries[:N_VAL]

test_articles = test_articles[:N_TEST]
test_summaries = test_summaries[:N_TEST]
```

Tokenizer: Converts words into integer indices

num_words = max_vocab: Limits vocabulary size

oov_token = : used to handle unseen or rare words safely without breaking the model.

Padding: Makes all sequences equal length

Decoder target: Trains model to predict next token

<pad) → padding token

<start) / <end) → sequence boundaries

```
#Tokenization
article_tokenizer = Tokenizer(num_words=max_vocab, oov_token='<unk>')
summary_tokenizer = Tokenizer(num_words=max_vocab, oov_token='<unk>')

article_tokenizer.fit_on_texts(train_articles)
summary_tokenizer.fit_on_texts(train_summaries)

def tokenize_and_pad(tokenizer, texts, max_len):
    seq = tokenizer.texts_to_sequences(texts)
    return pad_sequences(seq, maxlen=max_len, padding='post')
```

```
X_train = tokenize_and_pad(article_tokenizer, train_articles,
max_article_len)
X_val   = tokenize_and_pad(article_tokenizer, val_articles,
max_article_len)
X_test  = tokenize_and_pad(article_tokenizer, test_articles,
max_article_len)

y_train = tokenize_and_pad(summary_tokenizer, train_summaries,
max_summary_len)
y_val   = tokenize_and_pad(summary_tokenizer, val_summaries,
max_summary_len)
y_test  = tokenize_and_pad(summary_tokenizer, test_summaries,
max_summary_len)

decoder_target_train = np.expand_dims(y_train[:, 1], axis=-1)
decoder_target_val   = np.expand_dims(y_val[:, 1], axis=-1)
```

Bahdanau attention allows the decoder to dynamically focus on relevant encoder outputs instead of relying on a single fixed context vector.

W1 and W2: Learnable weight matrices to project encoder outputs and decoder hidden state

dec_hidden: Expanded to match encoder time steps

score: Measures relevance between encoder states and decoder state

softmax: Converts scores into attention weights

context: Weighted sum of encoder outputs used by the decoder

```
#Bahdanau attention layer
class BahdanauAttention(tf.keras.layers.Layer):
    def __init__(self, units):
        super().__init__()
        self.W1 = Dense(units)
        self.W2 = Dense(units)
        self.V = Dense(1)

    def call(self, enc_out, dec_hidden):
        dec_hidden = tf.expand_dims(dec_hidden, 1)
        score = self.V(tf.nn.tanh(self.W1(enc_out) +
self.W2(dec_hidden)))
        weights = tf.nn.softmax(score, axis=1)
        context = weights * enc_out
        return tf.reduce_sum(context, axis=1)
```

Luong attention uses dot-product similarity to focus on the most relevant encoder states during decoding.

dec_hidden: Expanded to align dimensions with encoder outputs

matmul: Computes dot-product similarity between encoder and decoder states

softmax: Converts similarity scores into attention weights

context: Weighted sum of encoder outputs used for prediction

```
#Luong attention layer
class LuongAttention(tf.keras.layers.Layer):
    def call(self, enc_out, dec_hidden):
        dec_hidden = tf.expand_dims(dec_hidden, 2)
        score = tf.matmul(enc_out, dec_hidden)
        weights = tf.nn.softmax(score, axis=1)
        context = weights * enc_out
        return tf.reduce_sum(context, axis=1)
```

This function creates an attention-based Seq2Seq model that generates summaries by learning to align input articles with output summaries.

Encoder: Converts the input article into contextual representations

Decoder: Generates the summary sequence using encoder states

Attention: Focuses on important parts of the input text

Concatenation: Combines context vector and decoder hidden state

Dense layer: Predicts the next word using softmax

Loss: Sparse categorical cross-entropy for word-level prediction

```
#Model building
def build_model(attention_type):
    enc_inputs = Input(shape=(max_article_len,))
    enc_emb = Embedding(max_vocab, embedding_dim)(enc_inputs)
    enc_out, enc_h, enc_c = LSTM(
    latent_dim,
    return_sequences=True,
    return_state=True)(enc_emb)

    dec_inputs = Input(shape=(max_summary_len,))
    dec_emb = Embedding(max_vocab, embedding_dim)(dec_inputs)
    _, dec_h, _ = LSTM(
    latent_dim,
    return_sequences=True,
    return_state=True)(dec_emb, initial_state=[enc_h, enc_c])

    if attention_type == 'bahdanau':
        context = BahdanauAttention(latent_dim)(enc_out, dec_h)
    else:
        context = LuongAttention()(enc_out, dec_h)

    concat = tf.keras.layers.concatenate([context, dec_h])
```

```
    outputs = Dense(max_vocab, activation='softmax')(concat)

    model = Model([enc_inputs, dec_inputs], outputs)
    model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy')
    return model
```

Early stopping prevents overfitting by stopping training when validation loss stops improving and restoring the best weights.

A higher patience value was used to allow the model sufficient time to learn before stopping.

monitor = 'val_loss' → Tracks validation loss

patience = 19 → Allows training to continue for 19 epochs without improvement

restore_best_weights = True → Reloads the best model weights

```
#Early stopping to avoid overfitting
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=19,
    restore_best_weights=True
)
```

This code trains the Bahdanau attention-based summarization model with early stopping to avoid overfitting.

build_model('bahdanau') → Initializes the Bahdanau attention-based model

X_train, y_train → Encoder and decoder inputs

decoder_target_train → Target next-word labels

validation_data → Used to monitor generalization

epochs & batch_size → Control training duration and efficiency

callbacks → Applies early stopping

The history object is used to track and visualize training and validation loss across epochs.

These values help:

Detect overfitting

Compare different models

Choose the best epoch

```
#Training model(Bahdanau)
bahdanau_model = build_model('bahdanau')
bahdanau_history = bahdanau_model.fit(
```

```
    [X_train, y_train], decoder_target_train,
    validation_data=([X_val, y_val], decoder_target_val),
    epochs=epochs,
    batch_size=batch_size,
    callbacks=[early_stop]
)
```

```
Epoch 1/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 27s 46ms/step - loss: 9.1409 - val_loss:
8.0885
Epoch 2/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 21s 45ms/step - loss: 7.6643 - val_loss:
8.1342
Epoch 3/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 21s 45ms/step - loss: 7.3482 - val_loss:
8.3399
Epoch 4/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 21s 45ms/step - loss: 7.0602 - val_loss:
8.6214
Epoch 5/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 21s 45ms/step - loss: 6.7723 - val_loss:
8.9883
Epoch 6/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 21s 44ms/step - loss: 6.4742 - val_loss:
9.1743
Epoch 7/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 21s 45ms/step - loss: 6.0579 - val_loss:
9.5411
Epoch 8/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 21s 44ms/step - loss: 5.6148 - val_loss:
9.9311
Epoch 9/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 21s 45ms/step - loss: 5.1383 - val_loss:
10.2077
Epoch 10/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 21s 45ms/step - loss: 4.6190 - val_loss:
10.3865
Epoch 11/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 41s 45ms/step - loss: 4.2565 - val_loss:
10.4197
Epoch 12/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 26s 56ms/step - loss: 3.6775 - val_loss:
10.6829
Epoch 13/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 30s 65ms/step - loss: 3.1291 - val_loss:
10.8845
Epoch 14/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 32s 45ms/step - loss: 2.6077 - val_loss:
11.1589
Epoch 15/20
```

```
469/469 ━━━━━━━━━━━━━━━━━━━━ 22s 46ms/step - loss: 2.1616 - val_loss:
11.3702
Epoch 16/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 22s 47ms/step - loss: 1.7615 - val_loss:
11.6490
Epoch 17/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 27s 57ms/step - loss: 1.4385 - val_loss:
11.8307
Epoch 18/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 35s 45ms/step - loss: 1.1073 - val_loss:
12.0920
Epoch 19/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 22s 46ms/step - loss: 0.8832 - val_loss:
12.3049
Epoch 20/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 21s 45ms/step - loss: 0.6887 - val_loss:
12.4982
```

This code trains the Seq2Seq model with Luong attention using training and validation data, with early stopping applied to prevent overfitting.

```
#Training model(Luong)
luong_model = build_model('luong')
luong_history = luong_model.fit(
    [X_train, y_train], decoder_target_train,
    validation_data=([X_val, y_val], decoder_target_val),
    epochs=epochs,
    batch_size=batch_size,
    callbacks=[early_stop]
)
```

```
Epoch 1/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 23s 44ms/step - loss: 9.1580 - val_loss:
8.0993
Epoch 2/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 20s 42ms/step - loss: 7.6247 - val_loss:
8.1359
Epoch 3/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 21s 43ms/step - loss: 7.2586 - val_loss:
8.4011
Epoch 4/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 21s 44ms/step - loss: 6.9106 - val_loss:
8.6584
Epoch 5/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 20s 43ms/step - loss: 6.5520 - val_loss:
9.0391
Epoch 6/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 20s 43ms/step - loss: 6.1262 - val_loss:
9.4554
```

```
Epoch 7/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 20s 43ms/step - loss: 5.6552 - val_loss:
9.9438
Epoch 8/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 21s 44ms/step - loss: 5.1355 - val_loss:
10.1525
Epoch 9/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 20s 42ms/step - loss: 4.5854 - val_loss:
10.6088
Epoch 10/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 20s 43ms/step - loss: 4.0337 - val_loss:
10.9358
Epoch 11/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 20s 42ms/step - loss: 3.4736 - val_loss:
11.3076
Epoch 12/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 20s 43ms/step - loss: 2.9742 - val_loss:
11.6363
Epoch 13/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 20s 42ms/step - loss: 2.5032 - val_loss:
11.9846
Epoch 14/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 20s 43ms/step - loss: 2.0708 - val_loss:
12.3148
Epoch 15/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 20s 42ms/step - loss: 1.6570 - val_loss:
12.6741
Epoch 16/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 21s 43ms/step - loss: 1.3197 - val_loss:
12.8925
Epoch 17/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 20s 42ms/step - loss: 1.0372 - val_loss:
13.1167
Epoch 18/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 20s 43ms/step - loss: 0.8013 - val_loss:
13.4328
Epoch 19/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 20s 42ms/step - loss: 0.6341 - val_loss:
13.6966
Epoch 20/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 20s 43ms/step - loss: 0.4860 - val_loss:
13.7630
```

The model summary is used to view the architecture and total parameters of the Bahdanau attention model.

```python
#Model parameter table for Bahdanau
print("Bahdanau Attention Model Summary")
bahdanau_model.summary()
```

Bahdanau Attention Model Summary

Model: "functional_2"

```
┏━━━
```

```
├──────────────────────┼──────────────────────┼──────────────┼──────────────────
──────┤
│ bahdanau_attention… │ (None, 128)          │       33,153 │ lstm_4[0][0],
│                      │                      │              │ lstm_5[0][1]
│                      │                      │              │
├──────────────────────┼──────────────────────┼──────────────┼──────────────────
──────┤
│ concatenate_2        │ (None, 256)          │            0 │
bahdanau_attenti… │
│ (Concatenate)        │                      │              │ lstm_5[0][1]
│                      │                      │              │
├──────────────────────┼──────────────────────┼──────────────┼──────────────────
──────┤
│ dense_11 (Dense)     │ (None, 50000)        │   12,850,000 │
concatenate_2[0]… │
└──────────────────────┴──────────────────────┴──────────────┴──────────────────
──────┘

 Total params: 77,838,965 (296.93 MB)

 Trainable params: 25,946,321 (98.98 MB)

 Non-trainable params: 0 (0.00 B)

 Optimizer params: 51,892,644 (197.95 MB)
```

The Luong model summary is used to analyze and compare its architecture and parameter count with the Bahdanau model.

```
#Model parameter table for Luong
print("\nLuong Attention Model Summary")
luong_model.summary()
```

```
Luong Attention Model Summary

Model: "functional_3"

┏━━━
```

```
│ embedding_6          │ (None, 400, 128)  │  6,400,000 │
input_layer_6[0]… │
│ (Embedding)          │                   │            │
│
├──────────────────────┼───────────────────┼────────────┼────────────────
──────┤
│ input_layer_7        │ (None, 50)        │          0 │ -
│
│ (InputLayer)         │                   │            │
│
├──────────────────────┼───────────────────┼────────────┼────────────────
──────┤
│ lstm_6 (LSTM)        │ [(None, 400,      │    131,584 │
embedding_6[0][0] │
│                      │ 128), (None,      │            │
│
│                      │ 128), (None,      │            │
│
│                      │ 128)]             │            │
│
├──────────────────────┼───────────────────┼────────────┼────────────────
──────┤
│ embedding_7          │ (None, 50, 128)   │  6,400,000 │
input_layer_7[0]… │
│ (Embedding)          │                   │            │
│
├──────────────────────┼───────────────────┼────────────┼────────────────
──────┤
│ lstm_7 (LSTM)        │ [(None, 50, 128), │    131,584 │
embedding_7[0][0… │
│                      │ (None, 128),      │            │ lstm_6[0][1],
│
│                      │ (None, 128)]      │            │ lstm_6[0][2]
│
├──────────────────────┼───────────────────┼────────────┼────────────────
──────┤
│ luong_attention      │ (None, 128)       │          0 │ lstm_6[0][0],
│
│ (LuongAttention)     │                   │            │ lstm_7[0][1]
│
├──────────────────────┼───────────────────┼────────────┼────────────────
──────┤
│ concatenate_3        │ (None, 256)       │          0 │
luong_attention[… │
│ (Concatenate)        │                   │            │ lstm_7[0][1]
│
├──────────────────────┼───────────────────┼────────────┼────────────────
──────┤
│ dense_12 (Dense)     │ (None, 50000)     │ 12,850,000 │
```

```
concatenate_3[0]… │
└────────────────────┴───────────────────┴──────────────┴──────────────────
──────┘

 Total params: 77,739,506 (296.55 MB)

 Trainable params: 25,913,168 (98.85 MB)

 Non-trainable params: 0 (0.00 B)

 Optimizer params: 51,826,338 (197.70 MB)
```

This function generates summaries one word at a time using greedy decoding until the end token is reached.

index_to_word → converts predicted token IDs back to words

startseq → signals the beginning of summary generation

model.predict() → predicts the next word

argmax → selects the most probable word (greedy decoding)

Loop stops at endseq or max length

Greedy decoding selects the most likely word at each step to generate the summary, without exploring alternative sequences.It may lead to repetitive or less coherent summaries because it does not look ahead to future word choices.

```
#Generating summary using token
index_to_word = summary_tokenizer.index_word
word_to_index = summary_tokenizer.word_index

def generate_summary(model, article_seq, max_len=30):
    summary = ['startseq']

    for _ in range(max_len):
        seq = summary_tokenizer.texts_to_sequences([' 
'.join(summary)])
        seq = pad_sequences(seq, maxlen=max_summary_len, 
padding='post')

        pred = model.predict([article_seq, seq], verbose=0)
        pred_id = np.argmax(pred)

        word = index_to_word.get(pred_id, None)
        if word is None or word == 'endseq':
            break

        summary.append(word)

    return ' '.join(summary[1:])
```

This function is used to visually compare generated summaries with reference summaries for qualitative evaluation.

Selects a few samples from the test dataset

Truncates the article for readability

Removes startseq and endseq from reference summaries

Generates summaries using the trained model

Prints article, reference summary, and generated summary side by side

```
#Output summaries using token
def show_samples(model, n_samples=3):
    for i in range(n_samples):
        print('=' * 80)
        print(f'SAMPLE {i+1}')

        original_article = test_articles.iloc[i][:500] + '...'
        reference_summary =
test_summaries.iloc[i].replace('startseq','').replace('endseq','')

        article_seq = X_test[i:i+1]
        generated_summary = generate_summary(model, article_seq)

        print('\nArticle:')
        print(original_article)

        print('\nReference Summary:')
        print(reference_summary)

        print('\nGenerated Summary:')
        print(generated_summary)
        print()

#Bahdanau attention result:
print('\nBahdanau Attention Results:')
show_samples(bahdanau_model, n_samples=2)
```

```
Bahdanau Attention Results:
================================================================================
SAMPLE 1

Article:
ever noticed how plane seats appear to be getting smaller and smaller
with increasing numbers of people taking to the skies some experts are
questioning if having such packed out planes is putting passengers at
risk they say that the shrinking space on aeroplanes is not only
uncomfortable it s putting our health and safety in danger more than
```

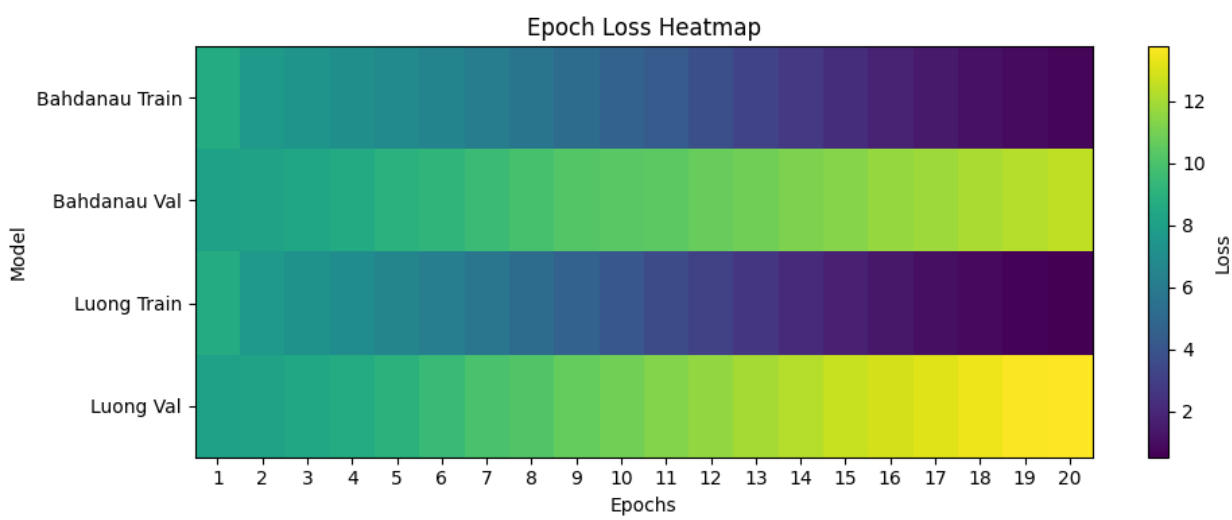
```
squabbling over the arm rest shrinking space on planes putting our
health and safety in danger this week a u s consumer advisory group
set up by the depart...

Reference Summary:
 experts question if packed out planes are putting passengers at risk
u s consumer advisory group says minimum space must be stipulated
safety tests conducted on planes with more leg room than airlines
offer

Generated Summary:
the the the the the the the the the the the the the the the the the the
the the the the the the the the the the the the the

================================================================
==========
SAMPLE 2

Article:
a drunk teenage boy had to be rescued by security after jumping into a
lions enclosure at a zoo in western india rahul kumar clambered over
the enclosure fence at the kamla nehru zoological park in ahmedabad
and began running towards the animals shouting he would kill them mr
kumar explained afterwards that he was drunk and thought i d stand a
good chance against the predators next level drunk intoxicated rahul
kumar climbed into the lions enclosure at a zoo in ahmedabad and began
running toward...

Reference Summary:
 drunk teenage boy climbed into lion enclosure at zoo in west india
rahul kumar ran towards animals shouting today i kill a lion
fortunately he fell into a moat before reaching lions and was rescued


Generated Summary:
the the the the the the the the the the the the the the the the the the
the the the the the the the the the the the the the


#Luong attention result:
print('\nLuong Attention Results:')
show_samples(luong_model, n_samples=2)


Luong Attention Results:
================================================================
==========
SAMPLE 1

Article:
ever noticed how plane seats appear to be getting smaller and smaller
```

```
with increasing numbers of people taking to the skies some experts are
questioning if having such packed out planes is putting passengers at
risk they say that the shrinking space on aeroplanes is not only
uncomfortable it s putting our health and safety in danger more than
squabbling over the arm rest shrinking space on planes putting our
health and safety in danger this week a u s consumer advisory group
set up by the depart...

Reference Summary:
 experts question if packed out planes are putting passengers at risk
u s consumer advisory group says minimum space must be stipulated
safety tests conducted on planes with more leg room than airlines
offer

Generated Summary:
the the the the the the the the the the the the the the the the the the
the the the the the the the the the the the the the the

==================================================================
==========
SAMPLE 2

Article:
a drunk teenage boy had to be rescued by security after jumping into a
lions enclosure at a zoo in western india rahul kumar clambered over
the enclosure fence at the kamla nehru zoological park in ahmedabad
and began running towards the animals shouting he would kill them mr
kumar explained afterwards that he was drunk and thought i d stand a
good chance against the predators next level drunk intoxicated rahul
kumar climbed into the lions enclosure at a zoo in ahmedabad and began
running toward...

Reference Summary:
 drunk teenage boy climbed into lion enclosure at zoo in west india
rahul kumar ran towards animals shouting today i kill a lion
fortunately he fell into a moat before reaching lions and was rescued


Generated Summary:
the the the the the the the the the the the the the the the the the the
the the the the the the the the the the the the the the
```

The epoch loss heatmap visually compares training and validation loss across epochs for both models to analyze learning behavior and overfitting.

```
#Heatmap of EPOCHS(Loss)

import numpy as np
import matplotlib.pyplot as plt
```

```
bah_train = bahdanau_history.history['loss']
bah_val   = bahdanau_history.history['val_loss']
lu_train  = luong_history.history['loss']
lu_val    = luong_history.history['val_loss']

heatmap_data = np.array([
    bah_train,
    bah_val,
    lu_train,
    lu_val
])

labels = [
    'Bahdanau Train',
    'Bahdanau Val',
    'Luong Train',
    'Luong Val'
]

epochs_range = np.arange(1, len(bah_train) + 1)

plt.figure(figsize=(10, 4))
plt.imshow(heatmap_data, aspect='auto', cmap='viridis')

plt.yticks(range(len(labels)), labels)
plt.xticks(range(len(epochs_range)), epochs_range)
plt.xlabel('Epochs')
plt.ylabel('Model')
plt.title('Epoch Loss Heatmap')
plt.colorbar(label='Loss')
plt.tight_layout()
plt.show()
```



ROUGE metrics are used to quantitatively compare generated summaries with reference summaries based on word overlap.

RougeScorer → Initializes ROUGE-1, ROUGE-2, and ROUGE-L metrics

use_stemmer=True → Improves matching by reducing words to their base form

model.predict() → Generates summary prediction

argmax → Selects the most probable predicted token

f-measure → Balanced score combining precision and recall

Average score → Final ROUGE value across test samples

ROUGE-2 is often zero because the model fails to generate matching word pairs with the reference summary.

ROUGE-1 measures word overlap, while ROUGE-L measures sequence-level similarity using the longest common subsequence.

Unigrams measure word overlap, while bigrams measure phrase-level overlap between generated and reference summaries.

Accuracy is not used because summarization does not have a single correct output and exact word-by-word matching is unrealistic.

```
#ROUGE evaluation
from rouge_score import rouge_scorer
scorer = rouge_scorer.RougeScorer(['rouge1','rouge2','rougeL'],
use_stemmer=True)

def compute_rouge(model, n=100):
    scores = {'rouge1': [], 'rouge2': [], 'rougeL': []}
    for i in range(n):
        pred = model.predict([X_test[i:i+1], y_test[i:i+1]],
verbose=0)
        pred_id = np.argmax(pred)
        pred_text = summary_tokenizer.sequences_to_texts([[pred_id]])
[0]
        ref_text = summary_tokenizer.sequences_to_texts([y_test[i]])
[0]
        s = scorer.score(ref_text, pred_text)
        for k in scores:
            scores[k].append(s[k].fmeasure)
    return {k: np.mean(v) for k,v in scores.items()}

bahdanau_rouge = compute_rouge(bahdanau_model)
luong_rouge = compute_rouge(luong_model)
```

This graph visually compares ROUGE scores of Bahdanau and Luong attention models to evaluate their summarization performance.

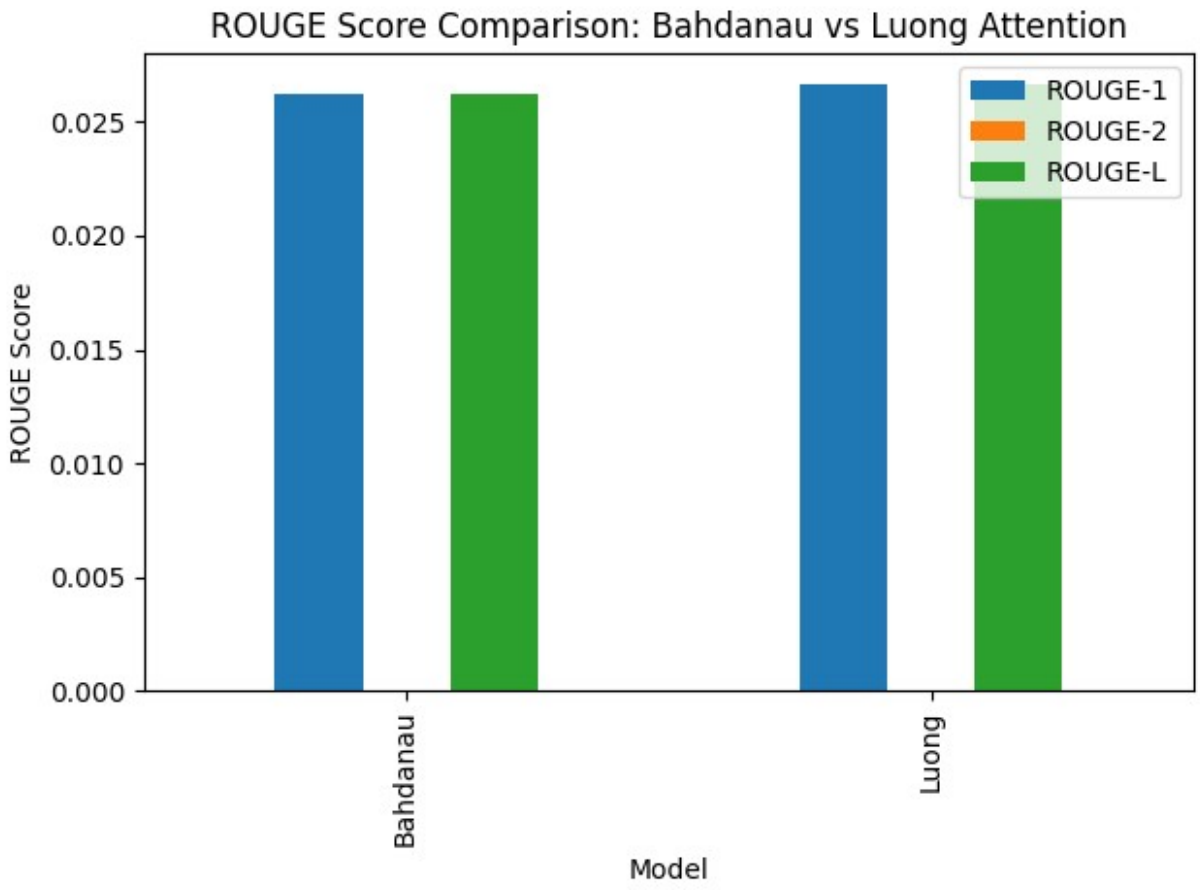
Creates a DataFrame containing ROUGE scores for both models

Sets the model names as the x-axis

Plots ROUGE-1, ROUGE-2, and ROUGE-L as grouped bars

Labels the axes and adds a title for clarity

```
#Rouge comparison graph
rouge_comparison = pd.DataFrame({
    'Model': ['Bahdanau', 'Luong'],
    'ROUGE-1': [
        bahdanau_rouge['rouge1'],
        luong_rouge['rouge1']
    ],
    'ROUGE-2': [
        bahdanau_rouge['rouge2'],
        luong_rouge['rouge2']
    ],
    'ROUGE-L': [
        bahdanau_rouge['rougeL'],
        luong_rouge['rougeL']
    ]
})

rouge_comparison.set_index('Model').plot(kind='bar')
plt.xlabel('Model')
plt.ylabel('ROUGE Score')
plt.title('ROUGE Score Comparison: Bahdanau vs Luong Attention')
plt.legend(loc='upper right')
plt.tight_layout()
plt.show()
```



The loss graph analyzes training behavior, and the ROUGE table compares summarization performance between Bahdanau and Luong models.

Loss plots → Show how model performance changes across epochs

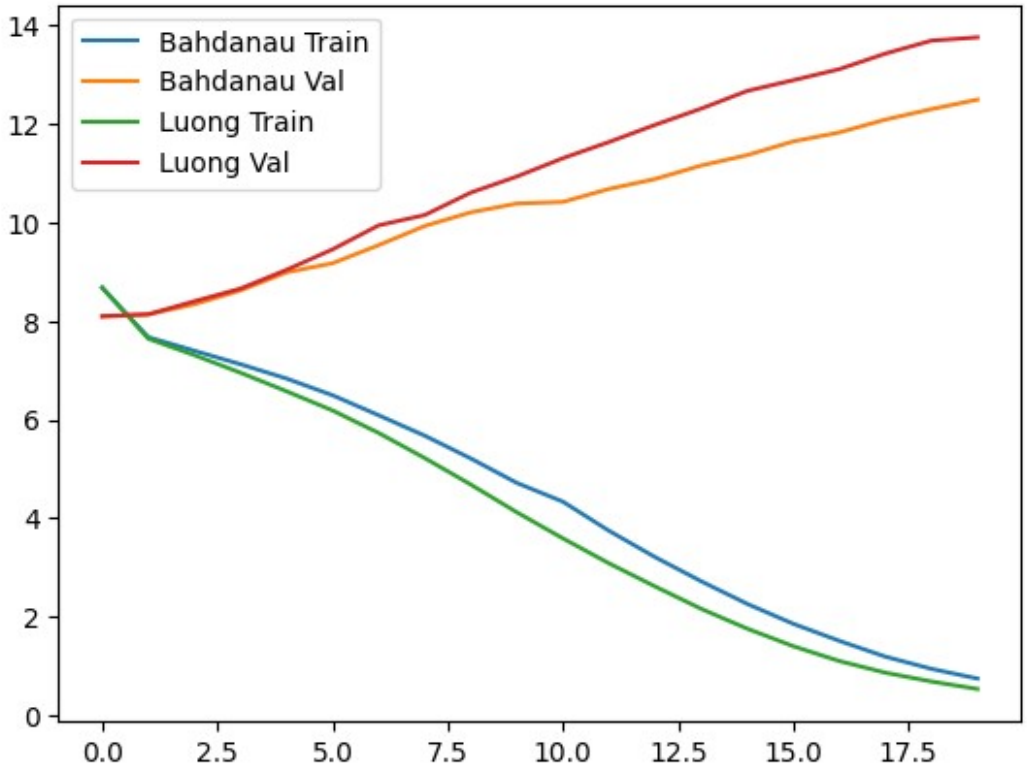
Training vs validation curves → Help identify overfitting

ROUGE table → Quantitatively compares summarization quality

Lower loss & higher ROUGE → Better model performance

```
#Plotting graphs
plt.figure()
plt.plot(bahdanau_history.history['loss'], label='Bahdanau Train')
plt.plot(bahdanau_history.history['val_loss'], label='Bahdanau Val')
plt.plot(luong_history.history['loss'], label='Luong Train')
plt.plot(luong_history.history['val_loss'], label='Luong Val')
plt.legend()
plt.show()

comparison = pd.DataFrame({
    'Model': ['Bahdanau', 'Luong'],
    'ROUGE-1': [bahdanau_rouge['rouge1'], luong_rouge['rouge1']],
```

```
    'ROUGE-2': [bahdanau_rouge['rouge2'], luong_rouge['rouge2']],
    'ROUGE-L': [bahdanau_rouge['rougeL'], luong_rouge['rougeL']]
})
print(comparison)
```




```
      Model   ROUGE-1  ROUGE-2   ROUGE-L
0  Bahdanau  0.026275      0.0  0.026275
1     Luong  0.026667      0.0  0.026667
```

This code uses a pre-trained T5 transformer model to generate high-quality abstractive summaries using beam search decoding.

T5Tokenizer → Converts text into tokens understood by T5

T5ForConditionalGeneration → Pre-trained transformer for text generation

"summarize:" prefix → Tells T5 the task is summarization

Beam search (num_beams=4) → Explores multiple word sequences for better summaries

length_penalty → Controls summary length

early_stopping → Stops when best summary is found

```
#Loading T5 model
from transformers import T5Tokenizer, T5ForConditionalGeneration
```

```
t5_tokenizer = T5Tokenizer.from_pretrained("t5-small")
t5_model = T5ForConditionalGeneration.from_pretrained("t5-small")

def t5_summarize(text, max_len=150):
    text = "summarize: " + text
    inputs = t5_tokenizer.encode(
        text, return_tensors="pt",
        truncation=True, max_length=512
    )
    summary_ids = t5_model.generate(
        inputs,
        max_length=max_len,
        min_length=40,
        num_beams=4,
        length_penalty=2.0,
        early_stopping=True
    )
    return t5_tokenizer.decode(summary_ids[0],
skip_special_tokens=True)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/
_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(
```

{"model_id":"efe10df8e0724dceab1d5cf35d3008fd","version_major":2,"version_minor":0}

{"model_id":"e296d9c003174981ba70110915179476","version_major":2,"version_minor":0}

{"model_id":"15f251bbbd7e4b2799d55d67867dbc4c","version_major":2,"version_minor":0}

```
You are using the default legacy behaviour of the <class
'transformers.models.t5.tokenization_t5.T5Tokenizer'>. This is
expected, and simply means that the `legacy` (previous) behavior will
be used so nothing changes for you. If you want to use the new
behaviour, set `legacy=False`. This should only be set if you
understand what it means, and thoroughly read the reason why this was
added as explained in
https://github.com/huggingface/transformers/pull/24565
```

{"model_id":"38b31afcdd434ed28d8a6aaf14676eea","version_major":2,"version_minor":0}

{"model_id":"ce0228762e74483b9db2f73e9a4e4f87","version_major":2,"version_minor":0}

{"model_id":"53781d6fda4f45429819317198643a85","version_major":2,"version_minor":0}

```
#Output summaries using T5
print("="*80)
print("Article:")
print(test_articles.iloc[0][:600])

print("\nT5 Generated summary:")
print(t5_summarize(test_articles.iloc[0]))

def show_t5_samples(n_samples=3):
    for i in range(n_samples):
        print("=" * 100)
        print(f"SAMPLE {i+1}")

        print("\nArticle:")
        print(test_articles.iloc[i][:600], "...")

        print("\nReference summary:")

print(test_summaries.iloc[i].replace('startseq','').replace('endseq',''))

        print("\nT5 Generated summary:")
        print(t5_summarize(test_articles.iloc[i]))
        print()

show_t5_samples(3)
```

```
================================================================================
Article:
ever noticed how plane seats appear to be getting smaller and smaller
with increasing numbers of people taking to the skies some experts are
questioning if having such packed out planes is putting passengers at
risk they say that the shrinking space on aeroplanes is not only
uncomfortable it s putting our health and safety in danger more than
squabbling over the arm rest shrinking space on planes putting our
health and safety in danger this week a u s consumer advisory group
set up by the department of transportation said at a public hearing
that while the government is happy to set standards

T5 Generated summary:
u.s consumer advisory group says shrinking space on planes is putting
```

our health and safety in danger more than squabbling over the arm rest shrinking space on planes. faa use planes with a inch pitch a standard which on some airlines has decreased many economy seats on united airlines have inches of room. united airlines has inches of space gulf air economy seats have between and inches air asia offers inches and spirit airlines offers just inches.
==========
SAMPLE 1

Article:
ever noticed how plane seats appear to be getting smaller and smaller with increasing numbers of people taking to the skies some experts are questioning if having such packed out planes is putting passengers at risk they say that the shrinking space on aeroplanes is not only uncomfortable it s putting our health and safety in danger more than squabbling over the arm rest shrinking space on planes putting our health and safety in danger this week a u s consumer advisory group set up by the department of transportation said at a public hearing that while the government is happy to set standards ...

Reference summary:
experts question if packed out planes are putting passengers at risk u s consumer advisory group says minimum space must be stipulated safety tests conducted on planes with more leg room than airlines offer

T5 Generated summary:
u.s consumer advisory group says shrinking space on planes is putting our health and safety in danger more than squabbling over the arm rest shrinking space on planes. faa use planes with a inch pitch a standard which on some airlines has decreased many economy seats on united airlines have inches of room. united airlines has inches of space gulf air economy seats have between and inches air asia offers inches and spirit airlines offers just inches.

==========
SAMPLE 2

Article:
a drunk teenage boy had to be rescued by security after jumping into a lions enclosure at a zoo in western india rahul kumar clambered over the enclosure fence at the kamla nehru zoological park in ahmedabad and began running towards the animals shouting he would kill them mr kumar explained afterwards that he was drunk and thought i d stand a good chance against the predators next level drunk intoxicated rahul kumar climbed into the lions enclosure at a zoo in ahmedabad and began running towards the animals shouting today i kill a lion mr kumar had been sitting near the enclosure when he sudd ...

```
Reference summary:
 drunk teenage boy climbed into lion enclosure at zoo in west india
rahul kumar ran towards animals shouting today i kill a lion
fortunately he fell into a moat before reaching lions and was rescued


T5 Generated summary:
rahul kumar climbed into a lions enclosure at the kamla nehru
zoological park in ahmedabad. he ran towards the animals shouting
today i kill a lion or a lion kills me. he fell into a moat as he ran
towards the lions and could be rescued by zoo security staff before
reaching the animals stock image.

==========================================================================
==============================
SAMPLE 3

Article:
dougie freedman is on the verge of agreeing a new two year deal to
remain at nottingham forest freedman has stabilised forest since he
replaced cult hero stuart pearce and the club s owners are pleased
with the job he has done at the city ground dougie freedman is set to
sign a new deal at nottingham forest freedman has impressed at the
city ground since replacing stuart pearce in february they made an
audacious attempt on the play off places when freedman replaced pearce
but have tailed off in recent weeks that has not prevented forest s
ownership making moves to secure freedman on a contract ...

Reference summary:
 nottingham forest are close to extending dougie freedman s contract
the forest boss took over from former manager stuart pearce in
february freedman has since lead the club to ninth in the championship


T5 Generated summary:
dougie freedman is set to sign a new two year deal at nottingham
forest freedman. the club s owners are pleased with the job he has
done at the city ground. freedman replaced stuart pearce but have
tailed off in recent weeks.
```